# IBM-Bob

# RT 周期負荷監視・診断スナップショット機能 基本設計書

## JSON 構造ファイル差し込み生成版

| 項目 | 内容 | JSON パス |
|---|---|---|
| **文書 ID** | BD-RTMON-001 | $.document_control.document_id |
| **対象機能 ID** | RTMON-001 | $.feature.feature_id |
| **対象システム** | IBM-Bob 基盤制御システム | $.system_context.system_name |
| **対象バージョン** | Ver.0.1 | $.document_control.target_version |
| **作成日** | 2026/04/27 | $.document_control.created_at |
| **作成者** | IBM-Bob AI Agent | $.document_control.author |
| **レビュー担当** | IBM-Bob Review Agent | $.document_control.reviewer |
| **レビュー状態** | draft | $.review_status |

本書は、IBM-Bob 基本設計書テンプレート v2.0 の章立てにい、sample_feature_rtmon_basic_design.json から各項目を差し込んで生成した評価用基本設計書である。

# 0. テンプレート利用方針

[SECTION_ID: BD-00] JSON パス: $

本書は、要件定義書から AI が抽出した JSON 構造ファイルを入力として、基本設計書テンプレートへ項目を挿入する運用を想定する。各章には安定した SECTION_ID と JSON パスを付し、逆方向の抽出およびレビュ AI によるプレレビューに利用する。

| 観点 | 方針 |
|---|---|
| JSON→Word 生成 | {{json.path}}相の差しみ規則により表箇書き文章を生成する。 |
| Word→JSON 抽出 | SECTION_ID と表見出しを手掛かりに、章単位で JSON へ再構造化する。 |
| AI プレレビュー | チェックリスト ID と JSON パスを突合し、記載漏れ・矛盾・高リスク箇所を抽出する。 |

# 1. 文書管理

[SECTION_ID: BD-01] JSON パス: $.document_control

| 項目 | 内容 | JSON パス |
|---|---|---|
| **文書名称** | RT 周期負荷監視・診断スナップショット機能 基本設計書 | $.document_control.title |
| **文書 ID** | BD-RTMON-001 | $.document_control.document_id |
| **対象バージョン** | Ver.0.1 | $.document_control.target_version |
| **作成日** | 2026/04/27 | $.document_control.created_at |
| **作成者** | IBM-Bob AI Agent | $.document_control.author |
| **承認者** | 未定 | $.document_control.approver |

| 版 | 改訂日 | 改訂内容 | 担当 |
|---|---|---|---|
| 0.1 | 2026/04/27 | JSON 構造ファイルから初版ドラフト生成 | IBM-Bob AI Agent |

# 2. 要件トレーサビリティ

[SECTION_ID: BD-02] JSON パス: $.requirements

| 要件 ID | 要件概要 | 設計方針 | 受入件 |
|---|---|---|---|
| REQ-RTMON-001 | リアルタイムスレッドの周期処理時間を監視し、閾値超過を診断できること。 | RT 側では測定値の更新と閾値判定のみ行い、ログ保存や文字列整形は TS 側に分離する。 | 模擬 RT 周期 1000 回で最大値、平均値、閾値超過回数が期待値と一致する。 |
| REQ-RTMON-002 | 外部診断アプリから API で監視状態を取得できること。 | 公開ヘッダに取得 API を追加し、既存 API の ABI 互換性は更しない。 | Api_GetRtCycleMonitorStatus()で現在値最大値超過回保存態を取得できる。 |
| REQ-RTMON-003 | 外部診断アプリまたは内部検知により、診スナップショットを CSV 保存できること。 | 保存要求はグローバル変数で TS 側へ依し、 RT 側でファイル I/O を行わない。 | 保存要求後、CSV ファイルにスレッド別統計と閾値超過履歴が出力される。 |
| REQ-RTMON-004 | 閾値超過が連続発生した場合に異常予兆フラグを共有メモリへ公開できること。 | RT 側で連続超過カウンタを持ち、警告フラグを立てる。警告の表示保存は TS 側で行う。 | 閾値超過が 3 周期連続した場合に warning_latched が ON になる。 |

# 3. 目的・背景・設計方針

[SECTION_ID: BD-03] JSON パス: $.purpose_background

| 項目 | 内容 | JSON パス |
|---|---|---|
| **目的** | 存基盤のリアルタイム周期余裕を見える化し、機能追加時の性能劣化やオーバラン予兆を早期検出できるようにする。 | $.purpose_background.purpose |
| **背景** | 評用基本設計書をカスタムエジェントへ入力した際、エージェントは既存仕様書およびコードへの影響調査を行う。この評価で | $.purpose_background.background |

| | | |
|---|---|---|
| | は、RT/TS 分担、グローバル変数、API 追加、状態管理、ログ保存の複合観点を含む仕追加が必要である。 | |
| **設計原則** | ・RT スレッドでは Windows API、ファイル I/O、動的メモリ確保、文字列整形を行わない。<br>・TS スレッドは保存要求を検知し、コピー済みデータを CSV へ出力する。<br>・外部公開 API は既存 API を変更せず、新規追加のみとする。<br>・共有データは所有者、読取元、書込元、更新周期を明示する。 | $.purpose_background.design_principles |

## 4. 対象範囲・非対象範囲

[SECTION_ID: BD-04] JSON パス: $.scope

| 項目 | 内容 | JSON パス |
|---|---|---|
| **対象範囲** | ・RT 周期処理時間の統計収集<br>・閾値超過回数と連続超過カウンタの管理<br>・診断スナップショット保存要求の受付<br>・TS 側 CSV 保存処理<br>・外部診断 API の追加<br>・単体テストおよび簡易機能テスト観点の定義 | $.scope.in_scope |
| **非対象範囲** | ・RT スケジューラそのものの優先度制御変更<br>・Windows の高精度タイマ設定変更<br>・外部 UI 面の新規作成<br>・クラウド送信やネットワーク連携 | $.scope.out_of_scope |
| **前提条件** | ・既存基盤には周期開始・周期終了付近にフック可能な関数が存在する。<br>・tick 計測は既存の軽量カウンタまたは性能測定用のラッパを使用できる。<br>・CSV 保存先は既存ログ保存ディレクトリ配下とする。 | $.scope.assumptions |

## 5. システム前提・既存構成

[SECTION_ID: BD-05] JSON パス: $.system_context

| 項目 | 内容 | JSON パス |
|---|---|---|
| **システム名** | IBM-Bob 基盤制御システム | $.system_context.system_name |
| **OS** | Windows | $.system_context.environment.os |
| **言語** | C | $.system_context.environment.language |
| コンパイラ | ・VC2026<br>・clang Windows 版 | $.system_context.environment.compiler_profiles |
| **共有メモリ** | 通信先システムへ公開される構造体定義済みグローバル変数を使用する。 | $.system_context.shared_memory |
| **基盤制約** | ・RT スレッドでファイル I/O、Windows システムコール、malloc/free、printf 系処理を行わない。<br>・グローバル変数の同時参照はロックレスを基本とし、更新順序と世代番号で整合性を確保する。<br>・公開ヘッダ変更時は構造体末尾への追加を原則とし、存フィルドのび型は更しない。 | $.system_context.constraints |

| スレッド名 | 種別 | 役割 | 制約 |
|---|---|---|---|
| RealTimeThread | リアルタイム | ハドウェア割により定周期で 1 回メインループを実行する。 | 全 RT 処理合計時間が 1 割込間隔を超えてはならない。 |
| TimeShareThread | タイムシェア | RT スレッド停止中の空き時間で、 | RT 用共有データへの更新は定められ |

| | | | |
|---|---|---|---|
| | | ファイル I/O、ログ出力、外部連携を行う。 | た要求/応答フラグ経由に限定する。 |

## 6. 機能概要

[SECTION_ID: BD-06] JSON パス: $.feature

| 項目 | 内容 | JSON パス |
|---|---|---|
| **機能 ID** | RTMON-001 | $.feature.feature_id |
| **機能名** | RT 周期負荷監視・診断スナップショット機能 | $.feature.name |
| **概要** | リアルタイムスレッドの周期処理時間を軽量に収集し、閾値超過時の統計値とスナップショットをタイムシェアスレッド経由で保存・外部 API 公開する。 | $.feature.summary |
| **優先度** | High | $.feature.priority |
| **想定追加規模** | 300～450 step | $.feature.estimated_new_steps |
| **関連機能** | ・基盤 RT スケジューラ<br>・タイムシェアワーカ<br>・診断 API<br>・イベントログ | $.feature.related_features |

## 7. 機能要件

[SECTION_ID: BD-07] JSON パス: $.functional_requirements

| ID | 機能要件 | 説明 | 主担当 |
|---|---|---|---|
| FR-001 | 周期処理時間測定 | 各 RT 周期において、対象 RT 処理の開始 tick と終了 tick の差分を測定し、最新値・最大値・累積値・周期数を更新する。 | RealTimeThread |
| FR-002 | 閾値超過判定 | 測定値が configured_threshold_tick を超過した場合、overrun_count および consecutive_overrun_count を加算する。 | RealTimeThread |
| FR-003 | 診断スナップショット要求 | API または内部閾値超過により保存要求を設定し、TS 側へ CSV 保存を依頼する。 | API/TimeShareThread |
| FR-004 | CSV 保存 | TS 側で統計情報と閾値超過履歴をローカルコピーして CSV 形式で保存し、保存結果コードを設定する。 | TimeShareThread |
| FR-005 | 外部 API 公開 | 診断アプリから状態取得・保存要求・カウンタクリアを行える API を提供する。 | API/TimeShareThread |

| コード/定数 | 値 | 説明 |
|---|---|---|
| RTMON_THREAD_MAX | 8 | 監視対象 RT スレッド数上限 |
| RTMON_OVERRUN_HISTORY_MAX | 32 | 閾値超過履歴保持件数 |
| RTMON_WARN_CONSEC_LIMIT | 3 | 警告ラッチ判定の連続超過回数 |

### 業務ルール・制御ルール

- 警告ラッチは連超過 3 回で ON にする。
- カウンタクリア API は RT 理中に直接クリアせず、clear_request を立て RT 周期境界で反映する。
- 保存要求中に追加要求がた場合は BUSY を返し、二重保存は行わない。

## 8. 利用シナリオ・運用シナリオ

[SECTION_ID: BD-08] JSON パス: $.scenarios

### SCN-001 通常監視

| 項目 | 内容 | JSON パス |
|---|---|---|
| **利用者/主体** | RealTimeThread | $.scenarios[].actor |
| **処理手順** | ・周期開始時に start_tick を取得する。<br>・周期終了時に end_tick を取得し、elapsed_tick を算出する。<br>・統計値を更新し、閾値未満なら正常継続する。 | $.scenarios[].steps |
| **例外** | tick 差分が不正な場合は当該周期の測定を無効化し、diagnostic_error_count を加算する。 | $.scenarios[].exception |

### SCN-002 閾値超過時のスナップショット保存

| 項目 | 内容 | JSON パス |
|---|---|---|
| **利用者/主体** | RealTimeThread/TimeShareThread | $.scenarios[].actor |
| **処理手順** | ・RT 側で elapsed_tick が閾を超過する。<br>・overrun_history へスレッド ID、elapsed_tick、tick 時刻を記する。<br>・連続超過が規定回数に達した場合、warning_latched を ON にする。<br>・auto_save_enabled が ON の場合、save_request を設定する。<br>・TS 側が save_request を検知し、CSV 保存を行う。 | $.scenarios[].steps |
| **例外** | 保存中に再度要求が発生した場合は pending_save_request を 1 にし、完了後に 1 回だけ再保存する。 | $.scenarios[].exception |

### SCN-003 外部診断アプリからの手動保存

| 項目 | 内容 | JSON パス |
|---|---|---|
| **利用者/主体** | 外部診断アプリ | $.scenarios[].actor |
| **処理手順** | ・Api_RequestRtCycleMonitorSave()を呼び出す。<br>・API は保存パスと要求番号を設定し、ACCEPTED を返す。<br>・TS 側が保存を実行し、結果コードを更新する。<br>・外部診断アプリは Api_GetRtCycleMonitorStatus()で完了を確認する。 | $.scenarios[].steps |
| **例外** | 保存パスが NULL または長すぎる場合、API は INVALID_ARGUMENT を返す。 | $.scenarios[].exception |

## 9. 状態遷移設計

[SECTION_ID: BD-09] JSON パス: $.state_machine

**状態遷移名: 診断スナップショット保存状態**

| 状態 ID | 説明 |
|---|---|
| RTMON_SAVE_IDLE | 保存要求なし |
| RTMON_SAVE_REQUESTED | API または内部検知により保存要求を受理 |
| RTMON_SAVE_COPYING | TS 側が共有データをローカルへコピー中 |
| RTMON_SAVE_WRITING | TS 側が CSV ファイルへ書込み中 |
| RTMON_SAVE_COMPLETE | 保存正常完了 |
| RTMON_SAVE_ERROR | 保存失敗 |

| 遷移元 | 遷移先 | トリガ |
|---|---|---|
| RTMON_SAVE_IDLE | RTMON_SAVE_REQUESTED | save_request=1 |
| RTMON_SAVE_REQUESTED | RTMON_SAVE_COPYING | TS 側が要求検知 |

| | | |
|---|---|---|
| RTMON_SAVE_COPYING | RTMON_SAVE_WRITING | コピー完了 |
| RTMON_SAVE_WRITING | RTMON_SAVE_COMPLETE | CSV 保存成功 |
| RTMON_SAVE_WRITING | RTMON_SAVE_ERROR | ファイルオープンまたは書込み失敗 |
| RTMON_SAVE_COMPLETE | RTMON_SAVE_IDLE | 外部取得または一定周期後 |
| RTMON_SAVE_ERROR | RTMON_SAVE_IDLE | エラー確認後 |

復初期化件: Api_ClearRtCycleMonitorCounters()実行時は保存中でなければ統計カウンタを RT 周期境界でクリアする。

# 10. シーケンス・応答動作設計

[SECTION_ID: BD-10] JSON パス: $.sequence_design

## SEQ-001 手動スナップショット保存シーケンス

| No | 主体 | 処理 | 応答/結果 |
|---|---|---|---|
| 1 | 外部診断アプリ | Api_RequestRtCycleMonitorSave(filePath)を呼び出す | ACCEPTED/BUSY/INVALID_ARGUMENT |
| 2 | API | filePath を内部バッファへコピーし save_request を ON にする | request_seq を加算 |
| 3 | TimeShareThread | save_request を検知する | save_state=COPYING |
| 4 | TimeShareThread | 共有統計をローカルバッファへコピーする | copy_generation を保持 |
| 5 | TimeShareThread | CSV ファイルを作成し書き込む | save_result を設定 |
| 6 | 外部診断アプリ | Api_GetRtCycleMonitorStatus()で結果を確認する | COMPLETE/ERROR |

## タイミング上の注意

- RT 側の 1 周期処理追加は監視対象 1 件あたり定数時間とする。
- CSV 保存処理は TS 側のみで実施し、RT 周期内では実行しない。

# 11. データ設計・共有メモリ・グローバル変数

[SECTION_ID: BD-11] JSON パス: $.data_design

| グローバル変数 | 型 | 所有者 | 書込元 | 読取元 | 説明 |
|---|---|---|---|---|---|
| g_rtCycleMonCtrl | RTMON_CTRL | RealTimeThread | ・RealTimeThread<br>・API<br>・TimeShareThread | ・RealTimeThread<br>・TimeShareThread<br>・外部診断 API | 周期負荷監視の統計、保存要求、保存状態を保持する制御構造体。 |

## 構造体: RTMON_THREAD_STAT

| フィルド | 型 | 説明 |
|---|---|---|
| thread_id | uint16_t | 監視対象スレッド ID |
| latest_tick | uint32_t | 最新周期処理時間 |
| max_tick | uint32_t | 最大周期処理時間 |
| sum_tick | uint64_t | 平均計算用累積値 |
| sample_count | uint32_t | 測定周期 |
| overrun_count | uint32_t | 値超過回 |

## 構造体: RTMON_CTRL

| フィルド | 型 | 説明 |
|---|---|---|
| enable | uint8_t | 監視有効フラグ |
| warning_latched | uint8_t | 連超過警告ラッチ |
| save_request | uint8_t | TS 側保存要求 |
| save_busy | uint8_t | 保存中フラグ |
| save_state | uint16_t | 保存状態 |
| configured_threshold_tick | uint32_t | 閾値 tick |
| stats[RTMON_THREAD_MAX] | RTMON_THREAD_STAT | スレッド別統計 |

### 整合性方針

- RT 側が stats を更新し、TS 側は保存前に generation_begin/generation_end を確認して整合コピーする。
- API からの要求フラグ設定は save_busy=0 かつ save_request=0 の場合のみ受け付ける。
- clear_request は RT 周期境界で反映し、TS 保存中は拒否または保留する。

## 12. 外部インターフェース・API 設計

[SECTION_ID: BD-12] JSON パス: $.interfaces

| API 名 | プロトタイプ | 説明 | 戻り値 |
|---|---|---|---|
| Api_GetRtCycleMonitorStatus | int32_t Api_GetRtCycleMonitorStatus(RTMON_STATUS* pStatus); | 現在の監視状態、統計サマリ、保存状態を取得する。 | ・API_OK<br>・API_ERR_INVALID_ARGUMENT |
| Api_RequestRtCycleMonitorSave | int32_t Api_RequestRtCycleMonitorSave(const char* pFilePath); | 診断スナップショット CSV 保存を要求する。 | ・API_OK<br>・API_ERR_INVALID_ARGUMENT<br>・API_ERR_BUSY |
| Api_ClearRtCycleMonitorCounters | int32_t Api_ClearRtCycleMonitorCounters(void); | 周期統計カウンタのクリアを要求する。 | ・API_OK<br>・API_ERR_BUSY |

| 公開ヘッダ | 変更内容 |
|---|---|
| include/ibm_bob_api_diag.h | RTMON_STATUS 構造体と API プロトタイプを追加 |
| include/ibm_bob_shared.h | RTMON_CTRL 構造体を末尾追加 |

| 外部システム | 方向 | 方式 | 認証/権限 |
|---|---|---|---|
| 外部診断アプリ | 呼び出し元 | C API | 既存プロセス権限に準拠 |

## 13. スレッド配置・処理分担

[SECTION_ID: BD-13] JSON パス: $.thread_allocation

| スレッド | 責務 | 禁止事項 |
|---|---|---|
| RealTimeThread | ・開始/終了 tick 差分測定<br>・統計更新<br>・閾値超過判定<br>・警告ラッチ設定<br>・clear_request 反映 | ・ファイル I/O<br>・文字列整形<br>・Windows API 直接呼び出し<br>・動的メモリ確保 |
| TimeShareThread | ・保存要求監視<br>・共有データ整合コピー<br>・CSV 保存<br>・保存結果コード更新<br>・イベントログ出力 | ・RT 出力信号の直接変更 |
| API | ・引数チェック<br>・保存要求受付<br>・状態コピー返却 | ・長時間待機<br>・RT 周期ブロック |

| From | To | データ | 方式 |
|---|---|---|---|
| API | TimeShareThread | save_request, save_path, request_seq | global |
| RealTimeThread | TimeShareThread | stats, overrun_history, warning_latched | global |
| TimeShareThread | API/外部診断アプリ | save_state, save_result | global |

## 14. リアルタイム性・性能・同期設計

[SECTION_ID: BD-14] JSON パス: $.non_functional

| 項目 | 内容 | JSON パス |
|---|---|---|
| **性能** | ・RT 側追加処理は対象スレッド 1 件あたり定数時間で完了すること。 | $.non_functional.performance |

| | | |
|---|---|---|
| | ・RT 処理追加時間の目安は 1 周期あたり 10usec 相当以下とする。<br>・CSV 保存は TS 側で分割実行し、1 回の TS 処理で過大なファイル I/O を継続しない。 | |
| **可用性** | ・監視機能が異常になっても機械制御は継続可能とする。<br>・CSV 保存失敗は制御停止ではなく診断エラーとして扱う。 | $.non_functional.availability |
| **保守性** | ・API、グローバル変数、エラーコード、ログ ID を台帳管理可能な ID 付きで定義する。<br>・VC2026/clang Windows 版の両方でビルドできる C89/C99 互換範の記法を採用する。 | $.non_functional.maintainability |

# 15. 異常処理・安全設計

[SECTION_ID: BD-15] JSON パス: $.error_safety_design

| エラー/警告コード | 条件 | 動作 |
|---|---|---|
| RTMON_ERR_INVALID_ARG | API 引 NULL またはパス長超過 | API_ERR_INVALID_ARGUMENT を返す |
| RTMON_ERR_SAVE_BUSY | 保存中に保存要求 | API_ERR_BUSY を返し、内部自動保存のみ pending 扱い可能 |
| RTMON_ERR_FILE_OPEN | CSV ファイル作成失敗 | save_state=ERROR, save_result にエラー設定 |
| RTMON_WARN_OVERRUN | 連続閾値超過 | warning_latched を ON、TS 側でイベントログ出力 |

## 安全側動作

- 本機能の異常は診断系異常として扱い、機械動作出力を直接変更しない。
- ただし、連続超過警告は既存インターロック判定の入力候補として公開する。採用可否は上位仕様で判断する。

# 16. ログ・診断・保守性

[SECTION_ID: BD-16] JSON パス: $.logging_diagnostics

| ログ ID | イベント | 出力項目 |
|---|---|---|
| LOG-RTMON-001 | 監視開始/停止 | ・tick<br>・enable<br>・threshold |
| LOG-RTMON-002 | 連続超過警告 | ・tick<br>・thread_id<br>・elapsed_tick<br>・threshold |
| LOG-RTMON-003 | スナップショット保存結果 | ・file_path<br>・result<br>・sample_count |

## スナップショット CSV 列

- timestamp
- thread_id
- latest_tick
- max_tick
- average_tick
- overrun_count
- warning_latched

# 17. セキュリティ・権限・公開範囲

[SECTION_ID: BD-17] JSON パス: $.security

| 項目 | 内容 | JSON パス |
|---|---|---|
| **公開範囲** | 診断アプリ向け公開 API。一般 UI からの直接呼び出しは想定しない。 | $.security.public_scope |
| パス**検証** | ・保存先は既存ログディレクトリ配下に限定する。<br>・相対パス上位参照（..）を拒否する。<br>・パス長上限を RTMON_PATH_MAX で制限する。 | $.security.path_validation |
| **データ保護** | CSV には個人情報を含めない。制御パラメータ値のみを出力する。 | $.security.data_protection |

# 18. 既存仕様書・ソースコード影響調査

[SECTION_ID: BD-18] JSON パス: $.impact_analysis

| ID | 種別 | 対象 | 影響内容 | 更要否 |
|---|---|---|---|---|
| IMP-001 | source | src/rt_scheduler.c | 周期開始/終了フックで測定 **闕**び出しを追加 | 要 |
| IMP-002 | source | src/ts_worker.c | 保存要求監視と CSV 保存処理を追加 | 要 |
| IMP-003 | source | src/api_diag.c | 診断 API 3 関数を追加 | 要 |
| IMP-004 | header | include/ibm_bob_shared.h | RTMON 構造および extern グローバル宣言を追加 | 要 |
| IMP-005 | header | include/ibm_bob_api_diag.h | 公開 API プロトタイプと RTMON_STATUS を追加 | 要 |
| IMP-006 | test | tests/test_rtmon.c | 統計更新、閾値判定、API 引数検証、保存状態遷移の単体テストを追加 | 要 |
| IMP-007 | document | 基盤システム_基本設計書 | 診断機能一覧と共有メモリーへ　RTMON を追記 | 要 |
| IMP-008 | build | workspace profile | VC2026/clang Windows 版の build/test コマンドでコンパイル確認 | 不要/確認 |

## リスクメモ

- RT スケジューラへのフック追加は周期負荷に直接影響するため、追加処理の境界と条件分岐数を詳細設計で確認する。
- 公開共有メモリ構造体の追加は通信先システムとのヘッダ同期が必要である。
- 保存パス検証不足は任意パス書込みにつながるため、API 層で制限する。

# 19. テスト観点・受入条件

[SECTION_ID: BD-19] JSON パス: $.test_acceptance

| 単体 ID | 対象 | 条件 | 期待結果 |
|---|---|---|---|
| UT-001 | RtMon_UpdateStat | elapsed=100, threshold=200 | overrun_count は増加しない |
| UT-002 | RtMon_UpdateStat | elapsed=250, threshold=200 | overrun_count が 1 加する |
| UT-003 | RtMon_UpdateStat | 3 回連続超過 | warning_latched が ON になる |
| UT-004 | Api_RequestRtCycleMonitorSave | 保存中に要求 | API_ERR_BUSY を返す |
| UT-005 | TS 保存態遷移 | 正常保存 | REQUESTED→COPYING→WRITING→COMPLETE |

| 機能 ID | シナリオ | 期待結果 |
|---|---|---|
| FT-001 | 外部診断アプリから手動保存を要求する | CSV が生成され、Api_GetRtCycleMonitorStatus() で COMPLETE が確認できる |
| FT-002 | 閾値超過を模擬して自動保存を発生させる | warning_latched が ON となり、イベントログが出力される |

## Build/Test コマンド

- cmd /c scripts\build_vc2026.bat
- cmd /c scripts\test_vc2026.bat

- cmd /c scripts\build_clang.bat
- cmd /c scripts\test_clang.bat

### 受入条件

- 全単体テストが成功すること。
- VC2026/clang Windows 版の両方でビルド成功すること。
- RT 側追加処理にファイル I/O、Windows API、malloc/free、printf 系処理が含まれないこと。
- 基本設計レビューで重大指摘が残っていないこと。

## 20. 実装範囲・見積・制約・未確定事項

[SECTION_ID: BD-20] JSON パス: $.implementation_plan

| 項目 | 内容 | JSON パス |
|---|---|---|
| 想定追加規模 | 300～450 step | $.implementation_plan.estimated_steps |
| 制約 | ・基本設計書では関数内部の詳細アルゴリズムは記載しすぎず、詳細設計へ委譲する。<br>・ただし RT 禁止事項と共有データ整合方針は基本設計で明示する。 | $.implementation_plan.constraints |

| モジュール | 内容 |
|---|---|
| rtmon.c | 統計更新・閾値判定・clear_request 処理 |
| ts_rtmon_save.c | CSV 保存状態遷移 |
| api_diag.c | 診断 API 追加 |
| test_rtmon.c | 単体テスト追加 |

| 未確定 ID | 内容 | 担当 |
|---|---|---|
| OPEN-001 | tick 計測の既存関数名と分解能は既存コード調査で確定する。 | 実装担当 |
| OPEN-002 | 自動保存の既定有効/無は運用と確認する。 | 設計担当 |

## 21. JSON 構造化仕様・Word 差し込み規約

[SECTION_ID: BD-21] JSON パス: $.appendix.json_mapping_policy

- 各章は[SECTION_ID: BD-xx]で固定し、JSON パスを併記する。
- 表形式の項目は JSON 配列として保持し、Word 生成時に行追加する。
- レビュ指摘は pre_review.findings へ構造化して戻す。

| 代表 JSON パス | 対応章 |
|---|---|
| $.requirements | 2. 要件トレサビリティ |
| $.functional_requirements | 7. 機能要件 |
| $.data_design.globals | 11. 共有メモリ・グローバル変数 |
| $.interfaces.public_api | 12. 外部インターフェース・API 設計 |
| $.impact_analysis.items | 18. 影響調査 |
| $.test_acceptance | 19. テスト観点・受入条件 |

## 22. AI プレレビュー用チェックリスト

[SECTION_ID: BD-22] JSON パス: $.pre_review

| チェック ID | 確認内容 |
|---|---|
| BRC-A01 | 要件 ID が原典と一致し、各要件に設計方針と受入条件があるか |
| BRC-A02 | RT スレッドで禁止処理が発生しない設計になっているか |
| BRC-A03 | TS スレッドへファイル I/O・ログ保存が分離されているか |
| BRC-A04 | グローバル変数の所有者・書込元・読取元が明示されているか |

| | |
|---|---|
| BRC-A05 | API 追加時に既存ヘッダ互換性を壊さないか |
| BRC-A06 | 状態遷移またはシーケンスの異常系が記載されているか |
| BRC-A07 | 既存仕様書・ソースコード影響調査が具体的なファイル単位で記載されているか |
| BRC-A08 | VC2026/clang Windows 版の build/test 確認観点があるか |

# 23. 付録

[SECTION_ID: BD-23] JSON パス: $.appendix

## 関連ドキュメント

- IBM-Bob_基本設計書テンプレート_v2.docx
- ibm_bob_basic_design_schema.json
- ibm_bob_pre_review_checklist.json
- 基盤システム_基本設計書.docx
- 基盤システム_詳細設計書.docx

## 付録 A: JSON 構造ファイル名

sample_feature_rtmon_basic_design.json